



Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



"Angular is a web framework that empowers developers to build fast, reliable applications that users love."

– <https://angular.dev/>

We have developed an Angular application in previous session, where Parent to Child and Child to Parent communication was done with following steps:

- Parent contains employee data.
- Parent sends employee data to Child using @Input().
- Child displays the employee.
- Child has a Promote button.
- When clicked, Child sends an event to Parent using @Output().
- Parent receives the event and updates the employee designation and salary.

Now we will extend the same application to understand Sibling → Sibling communication in Angular with Employee Management scenario.

The key concept is:

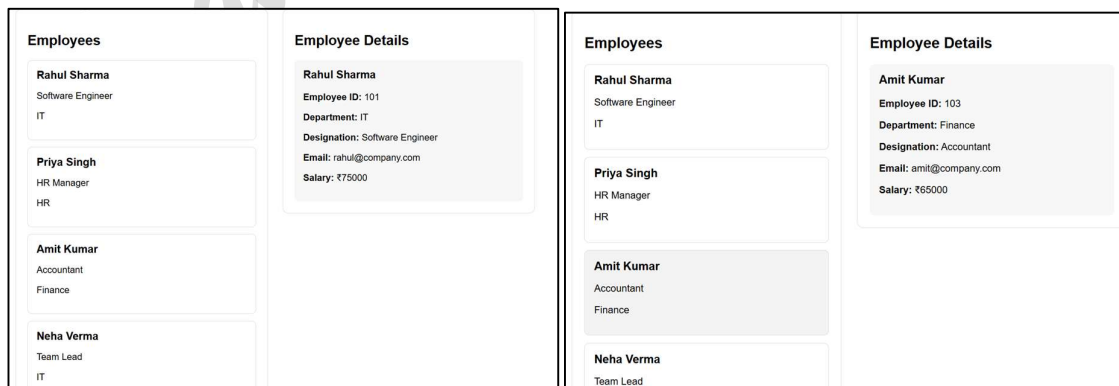
- Sibling components should not directly communicate with each other.
- Their common parent acts as the communication bridge.

We can build this scenario, imagine an Employee Management application.

- Employee List displays all employees.
- User clicks an employee in the list.
- EmployeeListComponent sends the selected employee to the Parent(empdashboard).
- Parent passes the selected employee to EmployeeDetailsComponent.
- Employee Details displays the selected employee details.

So, the communication flow is:

EmployeeList → Employee selected → EmpDashboard → Selected Employee → EmployeeDetails



 **Connecting parts, sharing data, building apps.** 



Presented by
Ranjan Bhatnagar



Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



Walkthrough:

Here I am creating a separate folder named components and will add three components as:

1. ng generate component empdashboard
2. ng generate component employee-list
3. ng generate component employee-details

Consider Employee Model used in previous walkthrough (Parent ↔ Child)

```
export interface EmpData {  
  id: number;  
  name: string;  
  department: string;  
  designation: string;  
  salary: number;  
  email: string;  
}
```

Consider parent component → empdashboard component

And two children or sibling components → employee-list and employee-details component

The parent is responsible for maintaining the selected employee.

Goto the empdashboard.ts and add

```
export class Empdashboard {  
  selectedEmployee: EmpData | null = null;  
  
  onEmployeeSelected(employee: EmpData) {  
    this.selectedEmployee = employee;  
  }  
}
```

Notice this property, initially, no employee is selected. When the child sends an employee, the parent stores it.

Goto the empdashboard.html and create template for view:

```
<div class="container">  
  <div class="content">  
    <!-- Sibling 1 -->  
    <app-employee-list  
      (employeeSelected)="onEmployeeSelected($event)">  
    </app-employee-list>  
    <!-- Sibling 2 -->  
    <app-employee-details [employee]="selectedEmployee">  
    </app-employee-details>  
  </div>  
</div>
```



Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



This is the heart of the example. Look carefully at these two lines in the code:

- The first one is: Child → Parent
- The second one is: Parent → Child
- Together they create: Sibling → Parent → Sibling

Now let's implement the first sibling. Goto employee-list.ts

```
employees: EmpData[] = [  
  {  
    id: 101,  
    name: 'Rahul Sharma',  
    department: 'IT',  
    designation: 'Software Engineer',  
    salary: 75000,  
    email: 'rahul@company.com'  
  },  
  {  
    id: 102,  
    name: 'Priya Singh',  
    department: 'HR',  
    designation: 'HR Manager',  
    salary: 85000,  
    email: 'priya@company.com'  
  },  
  {  
    id: 103,  
    name: 'Amit Kumar',  
    department: 'Finance',  
    designation: 'Accountant',  
    salary: 65000,  
    email: 'amit@company.com'  
  },  
  {  
    id: 104,  
    name: 'Neha Verma',  
    department: 'IT',  
    designation: 'Team Lead',  
    salary: 95000,  
    email: 'neha@company.com'  
  }  
];
```



Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



```
employeeSelected = output<EmpData>();
selectEmployee(employee: EmpData) {
  this.employeeSelected.emit(employee);
}
```

Here we're using the modern Angular output() API.

```
employeeSelected = output< EmpData >();
```

This creates an output event. When the user selects an employee:

```
this.employeeSelected.emit(employee);
```

we send the selected employee to the parent.

Now, create template for employee list, go to employee-list.html

```
<div class="employee-list">
  <h2>Employees</h2>
  @for (employee of employees; track employee.id) {
    <div
      class="employee-card" (click)="selectEmployee(employee)">
      <h3>{{ employee.name }}</h3>
      <p>{{ employee.designation }}</p>
      <p>{{ employee.department }}</p>
    </div>
  }
</div>
```

Now selected employee's information goes to the parent. Parent Receives the Employee and sends it to the Second Sibling i.e. Employee Details Component.

Goto employee-details.ts

Using modern Angular input(), EmployeeDetailsComponent expects employee data from its parent.

Design Employee Details Template, go to employee-details.html

```
<div class="employee-details">
  <h2>Employee Details</h2>
  @if (employee(); as emp) {
    <div class="details-card">
      <h3>{{ emp.name }}</h3>
      <p>
        <strong>Employee ID:</strong>{{ emp.id }}
      </p>
    </div>
  }
</div>
```





Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



```
<p>
  <strong>Department:</strong>{{ emp.department }}
</p>
<p>
  <strong>Designation:</strong>{{ emp.designation }}
</p>
<p>
  <strong>Email:</strong>{{ emp.email }}
</p>
<p>
  <strong>Salary:</strong>₹{{ emp.salary }}
</p>
</div>
}
@else {
  <p>Please select an employee from the list.</p>
}
</div>
```

Since employee is an Angular signal-based input, we read its value using: employee() as in @if (employee()); as emp) and then to view {{ emp.name }} etc.

Add css to style all components:

empdashboard.css

```
.container {
  width: 900px;
  margin: 30px auto;
  font-family: Arial, sans-serif;
}

h1 {
  text-align: center;
  margin-bottom: 30px;
}

.content {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 25px;
}
```





Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



employee-list.css

```
.employee-list {
  border: 1px solid #ddd;
  padding: 20px;
  border-radius: 10px;
}

.employee-card {
  border: 1px solid #ddd;
  padding: 15px;
  margin-bottom: 12px;
  border-radius: 8px;
  cursor: pointer;
}

.employee-card:hover {
  background-color: #f3f3f3;
}

.employee-card h3 {
  margin: 0 0 5px;
}
```

employee-details.css

```
.employee-details {
  border: 1px solid #ddd;
  padding: 20px;
  border-radius: 10px;
}

.details-card {
  padding: 15px;
  background-color: #f7f7f7;
  border-radius: 8px;
}

.details-card h3 {
  margin-top: 0;
}
```





Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



In app.html

```
<app-empdashboard></app-empdashboard>  
<router-outlet />
```

Test and Run then observe output.

🖥️🔗📁 Connecting parts, sharing data, building apps. 📁🔗🖥️

Angular by Ranjan B.



Presented by
Ranjan Bhatnagar

Page 7 / 7